

基于多种机器学习算法的网格导航动作预测方法比较研究

黄思敏 2024012703

摘要

路径规划是智能体导航中的基础问题。传统搜索算法能够在已知地图上给出最短路径，但每次面对新地图或新状态时均需要重新搜索。本文将二维网格导航转化为监督学习中的多分类任务：给定当前位置、目标位置以及局部障碍信息，预测智能体下一步动作。实验使用 BFS 作为专家策略自动生成标签，并构造两个难度不同的数据集：数据集 A 为 10×10 随机网格，数据集 B 为 $15 \times 15/20 \times 20$ 混合困难地图，其中包含更高障碍密度和窄通道结构。本文比较了贪心基线、KNN、距离加权 KNN、Gaussian Naive Bayes、逻辑回归、决策树、K-means 多数标签、Bagging 决策树、随机森林、AdaBoost 和 Stacking 等方法，并从单步动作预测、闭环导航和特征消融三个角度进行评价。

实验结果表明，单步动作预测上，Stacking 在数据集 A 测试集取得最高 Macro-F1 (0.8779)，随机森林在更困难的数据集 B 测试集取得最高 Macro-F1 (0.8176)。闭环导航中，Stacking 取得最好的综合成功率，在 A/B 测试集上分别达到 72.7% 和 62.8%。消融实验显示，相对位移与目标信息是动作预测的核心特征；单独去除距离标量、边界标志或地图密度对结果影响较小，而去除局部障碍与动作合法性相关信息会明显降低泛化表现。实验也揭示了单步准确率与闭环成功率之间并非完全一致：局部分类错误会在连续执行中累积，导致循环、碰撞或超时。因此，在网格导航动作预测任务中，应同时关注分类指标和环境闭环指标。

关键词：网格导航；动作预测；监督学习；BFS 专家；集成学习；特征消融；闭环评价

1 引言

二维网格导航问题通常被表述为：在含障碍物的离散地图中，智能体从起点出发，通过上、下、左、右四种动作到达目标位置。若完整地图已知，BFS 或 A* 等搜索算法可以求得最短路径。然而，在大量地图与状态上重复搜索会带来计算开销；同时，从机器学习角度看，专家路径中包含了可供模型学习的动作选择规律。因此，可以将路径规划问题转化为一个模仿学习式的监督分类问题：模型只观察当前位置、目标位置和局部障碍物特征，直接预测下一步动作。

这种转化具有两方面意义。第一，它提供了一个清晰、可控的数据生成方式：无需人工标注，专家动作由 BFS 最短路径自动产生。第二，它使得不同机器学习算法可以在统一任务中被横向比较，包括基于实例的方法、概率模型、线性模型、树模型、无监督聚类辅助分类和集成学习方法。与普通静态分类任务不同，导航任务还必须考虑闭环执行效果：一次错误动作可能使智能体偏离专家轨迹，并在后续状态中进入训练分布之外的区域。因此，本文不仅报告单步 Accuracy 与 Macro-F1，还将训练好的模型放回环境中连续执行，评价成功率、碰撞率、超时率和路径长度比。

2 任务定义与数据构造

2.1 网格环境与专家标签

设网格地图为 $G \in \{0, 1\}^{H \times W}$ ，其中 0 表示可通行单元格，1 表示障碍物。动作集合为

$$\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}.$$

在状态 $p = (r, c)$ 下，若执行动作后的位置仍在边界内且不是障碍，则该动作合法。给定起点 s 和目标 g ，实验使用 BFS 搜索一条最短路径

$$P = (p_0, p_1, \dots, p_L), \quad p_0 = s, \quad p_L = g.$$

路径中的每个中间状态展开为一个监督学习样本：

$$x_t = \phi(G, p_t, g), \quad y_t = a(p_t, p_{t+1}), \quad t = 0, \dots, L-1,$$

其中 ϕ 为特征提取函数， $a(p_t, p_{t+1})$ 为从 p_t 到 p_{t+1} 的专家动作。不可达地图、起终点相同或路径长度小于 1 的样本被丢弃。为了避免相邻状态同时出现在训练集和测试集导致过高估计，数据划分按地图进行，而不是按单个状态随机划分。

2.2 特征表示

仓库实现了 16 维状态特征。特征仅描述当前位置、目标和局部障碍结构，不把 BFS 的全局路径或最短路下一步作为模型输入，以避免标签泄漏。具体特征如表 1 所示。

表 1: 状态特征设计

特征组	含义
相对位移	dx, dy , 即目标行列坐标减去当前位置行列坐标
距离特征	曼哈顿距离 $ \Delta r + \Delta c $ 与欧式距离 $\sqrt{\Delta r^2 + \Delta c^2}$
障碍方向标志	<code>blocked_up, blocked_down, blocked_left, blocked_right</code> , 表示四个方向是否越界或被障碍阻挡
局部障碍统计	当前位置 3×3 与 5×5 窗口内的障碍数量
边界与密度	是否靠近地图边界, 以及整张地图的障碍物密度
动作合法性掩码	<code>legal_up, legal_down, legal_left, legal_right</code> , 表示四个动作是否合法

2.3 数据集设置

实验包含两个数据集。数据集 A 是较基础的 10×10 随机网格, 障碍物概率约为 0.10–0.30; 数据集 B 更困难, 包含 15×15 与 20×20 两种尺寸, 障碍物概率约为 0.20–0.40, 并以 0.35 的比例生成窄通道式地图。实际保存的数据统计如表 2 所示。

表 2: 数据集统计

数据集	地图数	样本数	特征数	训练样本	验证样本	测试样本	最短路均值
A	1500	11030	16	7668	1179	2183	7.35
B	900	11893	16	8255	1194	2444	13.21

两个数据集的动作类别总体较均衡。数据集 A 中 `up/down/left/right` 的样本数分别为 2819、2813、2656、2742; 数据集 B 中四类样本数分别为 2918、2941、2793、3241。数据集 B 的平均障碍密度为 0.2948, 高于数据集 A 的 0.1919, 且平均最短路径长度约为 A 的 1.8 倍, 因此可用于检验模型从简单随机网格到复杂地图的跨难度泛化能力。

3 模型与实验方法

3.1 比较算法

本文比较 10 个学习模型和 1 个非学习基线。贪心基线在每一步选择使曼哈顿距离下降最多的合法动作, 不进行训练; 它可以反映仅依赖局部目标方向时的性能上限与局限。学习模型包括:

- **KNN 与距离加权 KNN**: 基于历史相似状态投票, 后者按邻居距离赋权, 降低远邻样本影响。
- **Gaussian Naive Bayes**: 假设各维连续特征在类别条件下服从高斯分布, 并近似条件独立。
- **逻辑回归**: 学习多分类线性决策边界, 是计算开销小、可解释性较强的判别式模型。
- **决策树**: 通过递归特征划分学习非线性规则, 便于解释但可能过拟合。
- **K-means 多数标签**: 先对状态特征聚类, 再使用簇内多数动作作为预测标签, 用于观察无监督结构与动作类别的一致程度。
- **Bagging 决策树与随机森林**: 通过多树投票降低单棵树方差, 随机森林进一步引入特征随机化。
- **AdaBoost**: 使用 SAMME 多分类提升方法, 迭代关注误分样本。
- **Stacking**: 融合多个基学习器的输出, 再由逻辑回归作为二层模型做最终分类。

需要标准化的模型由 `StandardScaler` 预处理; 树模型直接使用原始特征。所有学习模型均在数据集 A 的训练集上训练, 在 A 的验证集上按 Macro-F1 选择超参数, 最终在 A/B 测试集上评价。最佳超参数如表 3 所示。

表 3: 验证集选择得到的最佳超参数

模型	最佳参数
knn	k=11, metric=manhattan
knn_weighted	k=11, metric=manhattan, weights=distance
naive_bayes	default
logreg	C=1.0
decision_tree	max_depth=6
kmeans_majority	clusters=32
bagging_tree	trees=100
random_forest	trees=80, max_depth=10
adaboost	iters=50, learning_rate=0.5
stacking	logreg + decision_tree + random_forest

3.2 评价指标

单步动作预测使用 Accuracy、Macro-F1、四类动作的 Precision/Recall、混淆矩阵、训练时间、预测时间和模型存储大小。设类别集合为 $\mathcal{C}$ ，则 Macro-F1 定义为

$$\text{MacroF1} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \text{F1}_c.$$

由于四类动作样本数接近均衡，Accuracy 与 Macro-F1 数值较接近，但 Macro-F1 更能反映各动作类别是否被均衡预测。

闭环导航评价将模型放回环境，从每张测试地图的起点开始连续预测动作，直到到达目标、发生碰撞或超过最大步数。最大步数设为 $4HW$ ，用于防止模型陷入无限循环。闭环指标包括：

$$\text{SuccessRate} = \frac{\text{成功到达目标的地图数}}{\text{测试地图总数}},$$

$$\text{PathRatio} = \frac{\text{模型成功路径长度}}{\text{BFS 最短路径长度}}.$$

同时记录平均步数、碰撞率和超时率。PathRatio 只对成功到达目标的 episode 计算。

实现上，`scripts/rollout_eval.py` 在每张测试地图上先对所有可通行位置批量计算模型动作，再在闭环执行时查表使用该动作。这样可以避免 KNN 等模型在循环状态中反复进行单样本预测，但不改变模型决策规则、终止条件或评价指标。

4 实验结果

4.1 单步动作预测结果

表 4 给出了数据集 A 测试集上的单步分类结果。Stacking 取得最高 Accuracy 和 Macro-F1，分别为 0.8781 和 0.8779；随机森林次之，Macro-F1 为 0.8680；逻辑回归、决策树、Bagging 和贪心基线都在 0.85 左右，说明该任务中相对位移和合法动作等特征已包含较强线性/规则性信号。K-means 多数标签明显落后，表明无监督聚类得到的簇结构并不能稳定对应专家动作类别。

表 4: 数据集 A 测试集单步动作预测结果

模型	参数	Accuracy	Macro-F1	训练/s	预测/s	大小/KB
stacking	lr_tree_rf	0.8781	0.8779	1.446	0.0367	7869.1
random_forest	trees=80,depth=10	0.8690	0.8680	0.271	0.0135	4679.1
logreg	C=1.0	0.8585	0.8585	0.084	0.0018	2.2
decision_tree	depth=6	0.8566	0.8555	0.036	0.0021	12.4
bagging_tree	trees=100	0.8548	0.8545	1.020	0.0220	14162.5
greedy	—	0.8520	0.8501	0.000	20.4023	0.0
adaboost	iters=50,lr=0.5	0.8401	0.8396	0.136	0.0063	35.8
knn_weighted	k=11,metric=manhattan,distance	0.8378	0.8378	0.001	0.1363	1020.0
knn	k=11,metric=manhattan	0.8314	0.8313	0.001	0.1352	1020.0
naive_bayes	default	0.7975	0.7978	0.002	0.0029	1.8
kmeans_majority	clusters=32	0.4544	0.4520	0.632	0.0264	35.8

表 5 展示了训练于数据集 A 后直接测试在数据集 B 上的跨难度泛化结果。整体指标相比 A 测试集下降，说明更大地图、更高障碍密度和窄通道结构带来了明显分布偏移。随机森林在 B 测试集上以 0.8176 的 Macro-F1 略高于 Stacking 的 0.8168，AdaBoost 与逻辑回归也保持在 0.81 左右。相比之下，Bagging、Naive Bayes 和 K-means 下降更明显。

表 5: 数据集 B 测试集单步动作预测结果

模型	Accuracy	Macro-F1
random_forest	0.8187	0.8176
stacking	0.8175	0.8168
adaboost	0.8167	0.8161
logreg	0.8126	0.8118
greedy	0.8106	0.8078
decision_tree	0.8065	0.8057
knn	0.7999	0.7992
knn_weighted	0.7942	0.7935
bagging_tree	0.7598	0.7579
naive_bayes	0.7586	0.7578
kmeans_majority	0.5086	0.4954

从复杂度看，逻辑回归的模型大小仅 2.2 KB，预测时间也很短，却取得了接近树模型的性能；KNN 训练几乎无需时间，但需要存储训练样本，模型大小约 1020 KB，预测阶段也显著更慢；Bagging 模型大小最大，超过 14 MB，但单步精度并未超过随机森林或 Stacking。贪心基线在表中预测时间较长，主要来自逐样本读取地图和用 Python 循环计算动作，而不是模型推理本身。

4.2 混淆矩阵分析

图 1 展示了代表性模型在 A 测试集上的混淆矩阵。Stacking 与随机森林的四个动作类别较均衡，主要错误来自左右/上下相近决策边界处的混淆；贪心基线在需要绕障时容易选择朝向目标但并非专家路径的动作；K-means 多数标签的预测分布则更分散，说明聚类中心难以表示精细的动作边界。

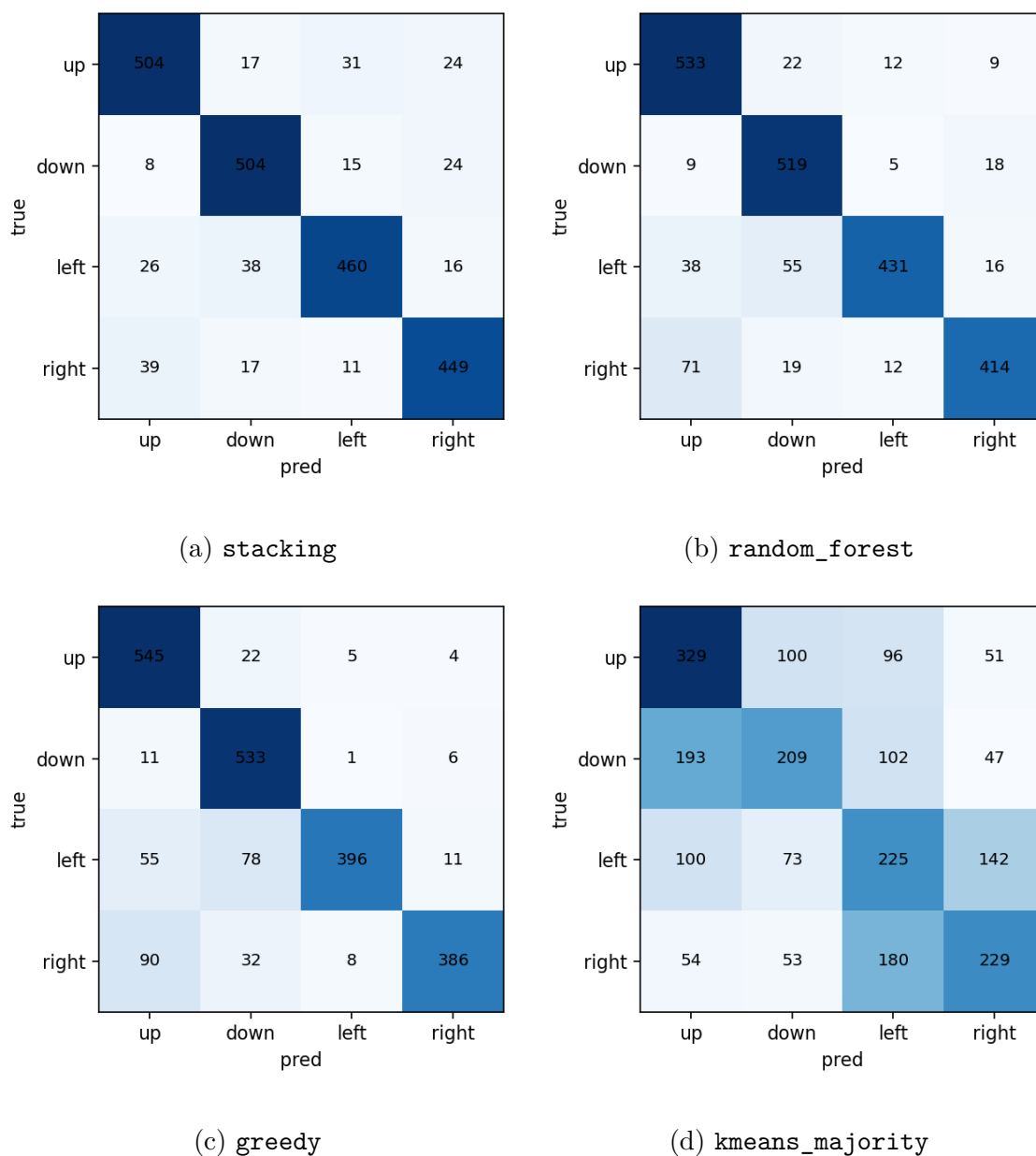


图 1: 代表性模型在数据集 A 测试集上的混淆矩阵

以 Stacking 为例，在 A 测试集上四类动作 Precision 分别为 0.8735、0.8750、0.8897、0.8752，Recall 分别为 0.8750、0.9147、0.8519、0.8702，整体较均衡。在 B 测试集上，Stacking 的 down 类 Recall 仍有 0.8758，但 up 和 right 的 Recall 分别下降到 0.7920

和 0.8040，说明复杂地图中不同方向的局部结构分布发生了变化。

4.3 闭环导航结果

表 6 汇总了各模型的闭环导航结果。闭环成功率与单步 Macro-F1 基本正相关，但并不完全一致。Stacking 在 A/B 测试集上均取得最高成功率，分别为 0.727 和 0.628；逻辑回归在闭环中非常有竞争力，A 测试集成功率为 0.723，B 测试集为 0.583，说明简单线性模型在该特征空间中具有较好的稳定性。随机森林在 B 测试集单步 Macro-F1 最高，但闭环成功率低于 Stacking 和逻辑回归，主要表现为较高超时率。

表 6: 闭环导航结果

模型	A 测试集				B 测试集			
	成功率	路径比	碰撞率	超时率	成功率	路径比	碰撞率	超时率
stacking	0.727	1.005	0.003	0.270	0.628	1.003	0.028	0.344
logreg	0.723	1.005	0.013	0.263	0.583	1.003	0.111	0.306
bagging_tree	0.693	1.012	0.033	0.273	0.533	1.025	0.078	0.389
random_forest	0.680	1.007	0.000	0.320	0.572	1.014	0.000	0.428
decision_tree	0.660	1.008	0.060	0.280	0.567	1.012	0.106	0.328
adaboost	0.640	1.006	0.240	0.120	0.489	1.004	0.344	0.167
knn_weighted	0.610	1.011	0.023	0.367	0.517	1.043	0.039	0.444
greedy	0.597	1.000	0.000	0.403	0.422	1.000	0.000	0.578
knn	0.597	1.025	0.037	0.367	0.478	1.018	0.050	0.472
naive_bayes	0.383	1.045	0.000	0.617	0.211	1.037	0.000	0.789
kmeans_majority	0.060	1.011	0.190	0.750	0.011	1.000	0.278	0.711

一个值得注意的现象是，所有成功 episode 的 PathRatio 都接近 1。这说明模型一旦能到达目标，其路径往往接近 BFS 最短路径；主要问题并不是绕路过长，而是在错误动作后进入循环、撞墙或超时。贪心基线的 PathRatio 为 1，是因为它只在能够沿曼哈顿距离下降到达目标的地图上成功；在需要短暂远离目标以绕过障碍时，贪心策略容易陷入局部最优并超时。AdaBoost 的超时率较低但碰撞率较高，说明它更倾向于做出激进动作，错误时更快失败。

4.4 特征消融结果

特征消融在每种特征删除设置下重新进行验证集调参，并在 A/B 测试集上评价。表 7 给出了各消融设置下 10 个学习模型的平均 Macro-F1、相对完整特征的平均变化，以及该设置下的最佳模型。贪心基线不参与消融，因为它不使用同一组学习特征。

表 7: 特征消融结果: 学习模型平均 Macro-F1 与最佳模型

消融设置	特征数	A 平均 F1	A 变化	B 平均 F1	B 变化	最佳模型 (A/B)
all	16	0.8073	0.0000	0.7672	0.0000	stacking / random_forest
no_relative_displacement	14	0.3955	-0.4118	0.3841	-0.3831	decision_tree / adaboost
no_distance	14	0.8152	+0.0079	0.7720	+0.0048	stacking / random_forest
no_goal_info	12	0.3678	-0.4395	0.3637	-0.4035	kmeans_majority / decision_tree
no_blocked_flags	12	0.8042	-0.0031	0.7679	+0.0007	stacking / random_forest
no_action_mask	12	0.8034	-0.0039	0.7670	-0.0002	stacking / stacking
no_local_obstacles	6	0.6779	-0.1294	0.6012	-0.1660	decision_tree / stacking
no_boundary	15	0.8124	+0.0051	0.7702	+0.0030	stacking / stacking
no_density	15	0.8080	+0.0007	0.7735	+0.0064	stacking / stacking

消融结果可以归纳为三点。第一，相对位移是最关键的特征。去掉 dx/dy 后，A 测试集平均 Macro-F1 从 0.8073 降到 0.3955；若进一步去掉所有目标相关信息，平均 Macro-F1 降到 0.3678。这说明动作预测首先需要知道目标相对当前位置的方向。第二，曼哈顿距离和欧式距离本身并非不可替代；去掉距离标量后结果没有下降，甚至略有提升，因为距离可以由 dx/dy 推出，直接加入距离可能带来冗余。第三，局部障碍信息对复杂地图尤其重要。去掉 blocked flags、legal mask 和局部窗口障碍数量后，B 测试集平均 Macro-F1 下降 0.1660，说明模型虽然知道目标方向，但无法可靠判断绕障动作。

单独去掉边界标志或地图密度时指标略有上升，这可能来自两种原因：一是这些特征与已有局部合法性特征存在重叠，二是训练于 A、测试于 B 时，全局密度分布发生变化，密度特征可能引入轻微分布偏移。总体而言，最稳健的特征组是相对位移加局部合法性/障碍结构；距离、边界和密度更像辅助特征。

5 讨论

5.1 单步分类与闭环控制的差异

导航动作预测不同于普通独立同分布分类任务。单步测试只评价模型在专家路径状态上的动作是否等于 BFS 选择的动作；但闭环执行时，模型可能进入专家路径以外的状态。即使单步错误率只有 10% 左右，连续几十步执行也可能积累为不可恢复的偏差。此外，BFS 最短路径并不唯一。在存在多条等长路径时，模型预测另一条可行最短路径的第一步会被单步指标计为错误，但闭环中可能仍能成功到达目标。因此，单步 Macro-F1 与闭环成功率相关但不等价。

5.2 模型差异解释

逻辑回归表现优于预期，说明当前特征空间中的主要决策规律接近线性：当目标在某个方向且该方向合法时，选择该方向通常是合理的；当被障碍阻挡时，再由局部障碍

和合法性特征修正。随机森林和 Stacking 能进一步捕获非线性绕障规则，因此单步指标更高。KNN 依赖样本邻近关系，在测试地图与训练地图分布相似时有效，但预测成本较高，且在更大地图上容易受尺度和局部结构差异影响。Naive Bayes 的条件独立假设与实际特征关系不符，例如 dx/dy 、距离和合法动作之间存在强相关，因此效果较弱。K-means 多数标签仅从无监督簇结构推断动作，无法充分利用标签边界，表现最差。

5.3 泛化难点

数据集 B 的主要困难来自地图尺度、障碍密度和窄通道结构。由于特征只包含局部窗口统计，模型无法直接观察远处通道连通性；当短期目标方向与长期可达路径冲突时，模型容易做出看似合理但最终导致循环的动作。闭环结果中较高的超时率正体现了这一点。若要进一步提高成功率，可以考虑以下改进：第一，在预测后加入动作合法性投影，强制排除非法动作；第二，收集闭环失败轨迹并加入训练，形成类似 DAgger 的数据迭代；第三，引入更大感受野的地图特征或图搜索启发特征；第四，将单步分类扩展为序列决策模型，显式优化到达率而非单步标签一致性。

6 结论

本文完成了一个基于机器学习的网格导航动作预测实验系统：通过 BFS 自动生成专家标签，以 16 维局部状态特征训练多种分类模型，并从单步预测、闭环导航和特征消融三方面进行评价。实验结果表明，Stacking 在综合闭环表现上最好，在 A/B 测试集成功率分别达到 72.7% 和 62.8%；随机森林在复杂数据集 B 的单步 Macro-F1 上最佳；逻辑回归以极小模型规模取得接近最优的闭环结果，是计算资源受限时的实用选择。相比之下，K-means 多数标签和 Naive Bayes 难以处理该任务中的非线性与特征相关性。

特征消融进一步证明，相对位移和目标方向信息是动作预测的核心，局部障碍与动作合法性信息决定模型能否绕开障碍并泛化到更复杂地图。单步准确率虽然重要，但不能完全代表闭环导航能力；在导航类任务中，应将环境交互评价作为最终性能判断依据。